



University of Alexandria

Faculty of Engineering

Division of Communications & Electronics

Subject: Microprocessors and Interfacing (CC 421)

Lecturer: Dr. Nayera Sadek

Semester: Spring 2026

SOFTWARE PROJECT — PROJECT 1

Tic-Tac-Toe Game

on the MTS-86C 8086 Microcomputer Trainer Kit

Team Members

#	Name	ID
1	Elsayed Ashraf Bakry	23010293
2	Mohamed Yasser Samak	24011107
3	Adham Khattab	24010991

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	Objectives	3
1.3	Tools and Environment	3
2	System Architecture	4
2.1	Segment and Memory Organisation	4
2.2	Hardware Connections Used	4
2.3	Software Modules (Subroutines)	5
3	Program Logic	6
3.1	Main Game Loop	6
3.2	Input Handling via 8279 Keypad Controller	6
3.3	Board Representation	7
3.4	Win Detection	8
3.5	Draw Detection	8
4	Hardware Integration	9
4.1	Overview	9
4.2	Keypad Input (8279 Controller)	9
4.3	LCD Display	9
4.4	LED Celebration (Win Only)	10
4.5	8255-3 Initialisation	10
5	Output and User Interface	11
5.1	Tera Term Terminal Output	11
5.2	Game Messages	11
6	Design Decisions	12
6.1	Key Design Decisions	12
6.2	Kit Simulation And Hardware Testing	13
7	Conclusion	15

A I/O Port Reference

16

1. Introduction

1.1 Project Overview

This report documents the design, implementation, and hardware integration of a two-player **Tic-Tac-Toe game** developed in Intel 8086 assembly language, running on the **MTS-86C Microcomputer Trainer Kit**. The project fulfils the requirements of Project 1 in the CC 421 Microprocessors and Interfacing Software Projects specification (Spring 2026), as set by Dr. Nayera Sadek.

The game allows two human players to alternate turns, placing their symbols (**X** and **O**) on a 3×3 grid. The program determines the winner when any player achieves three symbols in a row, column, or diagonal, or declares a draw when all nine cells are filled with no winner.

1.2 Objectives

- Implement a fully functional two-player Tic-Tac-Toe game in 8086 assembly language.
- Display the game board and all messages via RS-232 to a Tera Term terminal session.
- Accept all player input through the **physical kit keypad** (8279 controller).
- Provide hardware feedback on game end using the **kit LCD** and **LED bank**.

1.3 Tools and Environment

Hardware	MTS-86C 8086 Microcomputer Trainer Kit
Assembler	MASM (Microsoft Macro Assembler)
Terminal	Tera Term — RS-232 serial link to the kit
Language	Intel 8086 Assembly (flat COM-style segment)
Simulation	emu8086 / DOSBox for pre-lab verification

2. System Architecture

2.1 Segment and Memory Organisation

The entire program resides in a single `CODE` segment assembled at `ORG 0`. The segment register `DS` is initialised to equal `CS` at startup, so all data labels and code labels share the same physical segment — eliminating any need for cross-segment fixups and allowing `EXE2BIN` conversion.

The stack pointer is set to `SP = 4000H`, placing the runtime stack safely above the data area and well within the 64 KB segment boundary.

Data variables (game grid, player symbol, move counter, display strings) are placed *after* all procedure definitions. This ensures the CPU never falls through into the data area and cannot misinterpret ASCII bytes as opcodes.

2.2 Hardware Connections Used

Table 2.1: MTS-86C I/O Ports Used by the Program

Port Address	Device	Direction	Function
3FD6H	8255-3 CWR	OUT	Configure PPI: Port B = output (LEDs)
3FD2H	8255-3 Port B	OUT	Drive 8 LEDs (win celebration)
FFEAH	8279 Status	IN	Poll keypad FIFO Data Available flag
FFE8H	8279 Data	IN	Read key scan code from FIFO
FFC5H	LCD Data	OUT	Write character data to LCD DDRAM
FFC3H	LCD Status	IN	Read LCD Busy Flag before each write
FFC1H	LCD Command	OUT	Send initialisation/control commands to LCD

2.3 Software Modules (Subroutines)

Table 2.2: Subroutine Summary

Subroutine	Returns	Purpose
DRAW_BOARD	—	Renders the full 3×3 grid to Tera Term after every move
CHECK_WIN	AL	Checks all 8 winning lines; AL=1 if current player wins
HARDWARE_OUTPUT	never	Infinite left-rotating LED pattern (win celebration)
LCD_WAIT	—	Polls LCD Busy Flag until the controller is ready
LCD_CMD_WRITE	—	Sends one command byte to the LCD
LCD_DAT_WRITE	—	Sends one data byte (character) to the LCD
LCD_WIN_MSG	—	Writes “PLAYER X WINS!” and “CONGRATULATIONS” to the LCD
LCD_DRAW_MSG	—	Writes “DRAW GAME!!” and “WELL PLAYED” to the LCD

3. Program Logic

3.1 Main Game Loop

The program follows a strict sequential game-loop structure:

1. **Initialise hardware** — set 8255-3 control word (90H), clear LEDs, display welcome banner.
2. **Draw the board** — call `DRAW_BOARD` to render the current grid state to Tera Term.
3. **Prompt the player** — print “Player X/O, press keypad key (1–9):”.
4. **Read and validate input** — poll the 8279 keypad controller; reject invalid scan codes.
5. **Check cell occupancy** — reject the move if the chosen cell already holds X or O.
6. **Place the mark** — write the current player symbol into the `GRID` array; increment `MOVES`.
7. **Check for win** — call `CHECK_WIN`; if `AL = 1`, branch to `GAME_WIN`.
8. **Check for draw** — if `MOVES = 9` with no winner, branch to `GAME_DRAW`.
9. **Switch player** — toggle `PLAYER` between 'X' and 'O'; return to step 2.

3.2 Input Handling via 8279 Keypad Controller

Player input is read entirely from the physical kit keypad, *not* from a PC keyboard. The 8279 Keyboard/Display Controller maintains an 8-entry key FIFO. The program polls the 8279 Status Register at port `FFEAH`: bit 0 becomes 1 when at least one key is available in the FIFO. The scan code is then popped by reading port `FFE8H`.

Scan code mapping for the MTS-86C keypad:

Physical Key	Scan Code (hex)	Grid Cell Selected
1	01H	1
2	02H	2
⋮	⋮	⋮
9	09H	9
0 / others	00H or >09H	<i>Rejected — re-prompt</i>

Adding 30H ('0') to the scan code converts it to the ASCII digit (01H + 30H = 31H = '1', and so on). Scan code 00H (key "0") and any code above 09H are rejected, re-printing the prompt and waiting again.

The validated ASCII character is echoed back to the Tera Term terminal using INT 21H / AH=02H so the player can see their selection.

3.3 Board Representation

The 9-cell board is stored as a linear byte array `GRID`, initialised to the ASCII characters '1' through '9'. A free cell retains its digit; a claimed cell is overwritten with 'X' or '0' by the `PLACE_MARK` routine.

`DRAW_BOARD` alternates between printing fixed frame strings (via INT 21H / AH=09H) and printing individual live cell bytes (via AH=02H). This interleaving approach avoids any runtime string-building and keeps the code tight.

An example board mid-game might display as:

```

+-----+-----+-----+
| | | |
| X | 2 | 0 |
| | | |
+-----+-----+-----+
| | | |
| 4 | X | 6 |
| | | |
+-----+-----+-----+
| | | |
| 7 | 8 | 9 |
| | | |
+-----+-----+-----+

```

3.4 Win Detection

CHECK_WIN tests all eight possible winning combinations using a short-circuit comparison strategy. The current player's symbol is loaded into register CH once. For each combination, three consecutive CMP/JNZ pairs check the three relevant cells. The first mismatch skips immediately to the next combination, avoiding unnecessary comparisons. If all three cells of any combination match, the subroutine returns AL = 1; otherwise AL = 0.

The eight combinations checked are:

- **Rows:** [0,1,2] [3,4,5] [6,7,8]
- **Columns:** [0,3,6] [1,4,7] [2,5,8]
- **Diagonals:** [0,4,8] [2,4,6]

3.5 Draw Detection

A move counter MOVES is incremented each time a mark is placed. After CHECK_WIN returns 0, the program compares MOVES against 9. If all nine cells are filled with no winner, the game is declared a draw.

4. Hardware Integration

4.1 Overview

Extending the simulation-only version to interact with three physical peripherals on the MTS-86C kit: the **keypad**, the **LCD display**, and the **LED bank**. All three were integrated into the final submitted assembly file.

4.2 Keypad Input (8279 Controller)

As described in Chapter 3, the entire input path was rewritten to poll the 8279 controller rather than using `INT 21H / AH=01H`. This was the primary modification needed to run the game interactively on the physical kit without a connected PC keyboard.

The polling loop is non-blocking from the game-logic perspective — it simply waits at `KBD_POLL` until the 8279 reports a key in its FIFO, then reads and validates the scan code before returning control to the main loop.

4.3 LCD Display

The LCD (connected at ports `FFC1H–FFC5H`) displays a concise result message when the game ends, complementing the longer Tera Term output.

Table 4.1: LCD Output Messages

Game Outcome	LCD Line 1	LCD Line 2
Player X wins	PLAYER X WINS!	CONGRATULATIONS!
Player O wins	PLAYER O WINS!	CONGRATULATIONS!
Draw	** DRAW GAME!! **	WELL PLAYED!

Each LCD write follows the mandatory busy-wait sequence: `LCD_WAIT` polls bit 7 of the status port (`FFC3H`) until the Busy Flag clears before every command or data byte is sent. The LCD is re-initialised (8-bit mode, 2 lines, display on, cursor off, clear) at the start of each message to ensure a clean starting state regardless of any leftover content.

4.4 LED Celebration (Win Only)

On a win, after the LCD message is written, `HARDWARE_OUTPUT` is called. This routine drives an infinite left-rotating single-LED pattern on 8255-3 Port B (3FD2H):

- One LED is lit at a time, scanning from L1 (bit 0) to L8 (bit 7) and wrapping back.
- The rotation is implemented with `ROL AL, 1` — the carry wraps bit 7 back to bit 0.
- A software delay (`MOV CX, 0A000H / LOOP $`) produces a visible scan rate, matching the technique from the Lab Manual Exercise 2.
- The routine never returns — it is an infinite loop, appropriate since the game is over and the CPU should remain active until the kit is reset.

On a draw, no LED pattern is shown; the program proceeds cleanly to the exit prompt.

4.5 8255-3 Initialisation

Before any LED output, the 8255-3 PPI is configured by writing control word `90H` to the Control Word Register at `3FD6H`. The bit breakdown of `90H` (`1001 0000B`) is:

Bit	D7	D6	D5	D4	D3	D2
Value	1	0	0	1	0	0
Meaning	I/O mode	Group A: Mode 0		Port A = IN	Port C _{upper} = OUT	Group B: Mode 0

Port B as output allows the LED patterns to be driven directly by `OUT DX, AL`.

5. Output and User Interface

5.1 Tera Term Terminal Output

All in-game communication with the players is performed through the RS-232 link to a Tera Term terminal window running on the connected PC. The program uses exclusively INT 21H DOS interrupts for this purpose:

- **AH = 09H** — print a \$-terminated string (board frames, banners, messages).
- **AH = 02H** — print a single character from register DL (live cell content, player symbol in prompts and win messages).
- **AH = 01H** — read one character (used only for the final “press any key to exit” pause).
- **AH = 4CH** — terminate program and return to the MTS-86C monitor.

5.2 Game Messages

Message Sequence During a Game

1. **Welcome Banner** — displayed once at startup; shows player assignments (X / O) and keypad instructions.
2. **Board Display** — redrawn before every prompt; shows the current state of all nine cells.
3. **Player Prompt** — “Player X/O, press keypad key (1–9):”.
4. **Invalid Input** — re-prints the prompt silently if an out-of-range key is pressed.
5. **Cell Taken Error** — “Cell already taken! Try again.” if an occupied cell is chosen.
6. **Win Announcement** — “Player X WINS! Congratulations!” (with final board).
7. **Draw Announcement** — “DRAW GAME! Well played, both!” (with final board).
8. **Exit Prompt** — “Press any key to exit...”

6. Design Decisions

6.1 Key Design Decisions

Flat single-segment model. Using a single `CODE` segment with `DS = CS` simplifies all address resolution and avoids segment prefix complexity. It is the standard model for MTS-86C programs, consistent with all lab manual examples.

Data placed after code. Variables and strings are defined after all `ENDP` directives. If placed at `ORG 0`, the CPU would execute string bytes as opcodes on startup. The current placement guarantees that no execution path falls through into the data area.

Keypad polling over interrupt-driven input. A busy-wait polling loop on the 8279 status register was chosen over interrupt-driven input because the 8086 kit's interrupt vectors are partly occupied by the monitor firmware. Polling is simpler, deterministic, and fully sufficient for a human-paced game.

Draw detection by move counter. Counting moves is cheaper than re-scanning the entire grid for empty cells after each turn. Since every valid move increments the counter exactly once, `MOVES = 9` with no win is a guaranteed draw.

Infinite LED loop on win. The LED celebration routine never returns — it loops until the kit is manually reset. This is appropriate because the game is over and there is nothing further for the CPU to do. The pattern (rotating single LED) is unambiguous as a victory indicator.

LCD busy-wait before every write. The LCD busy-flag poll is called before every command and every data byte. While slower than a fixed-delay approach, it is more robust across different LCD controller speeds and avoids garbled characters caused by writing to a busy controller.

6.2 Kit Simulation And Hardware Testing

The assembled binary was loaded into the MTS-86C kit via the monitor's upload facility. Both players entered moves via the physical numeric keypad. The Tera Term terminal correctly displayed all board states and messages. On game end, the LCD showed the result message and the LED bank produced the rotating celebration pattern.

The following photographs document the program running live on the MTS-86C kit.

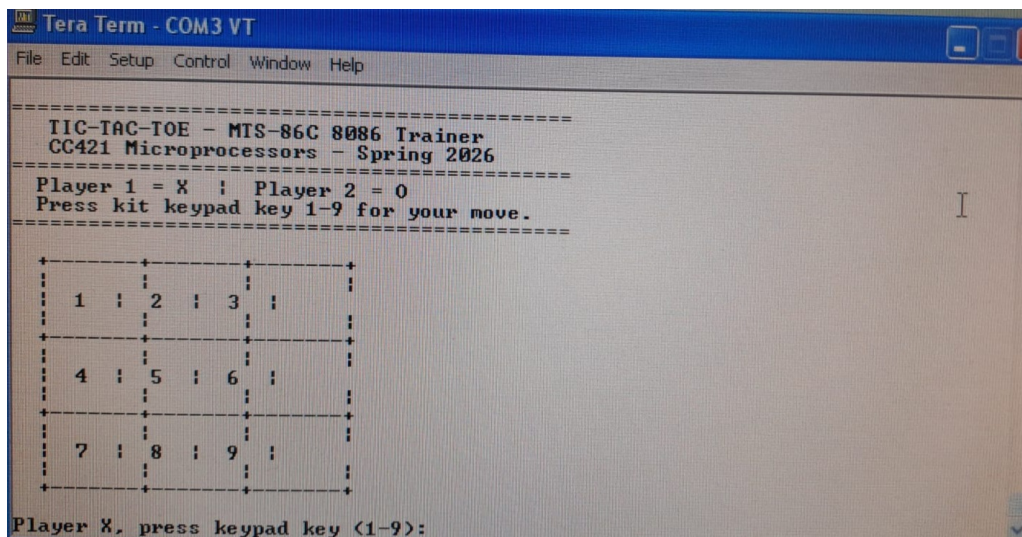


Figure 6.1

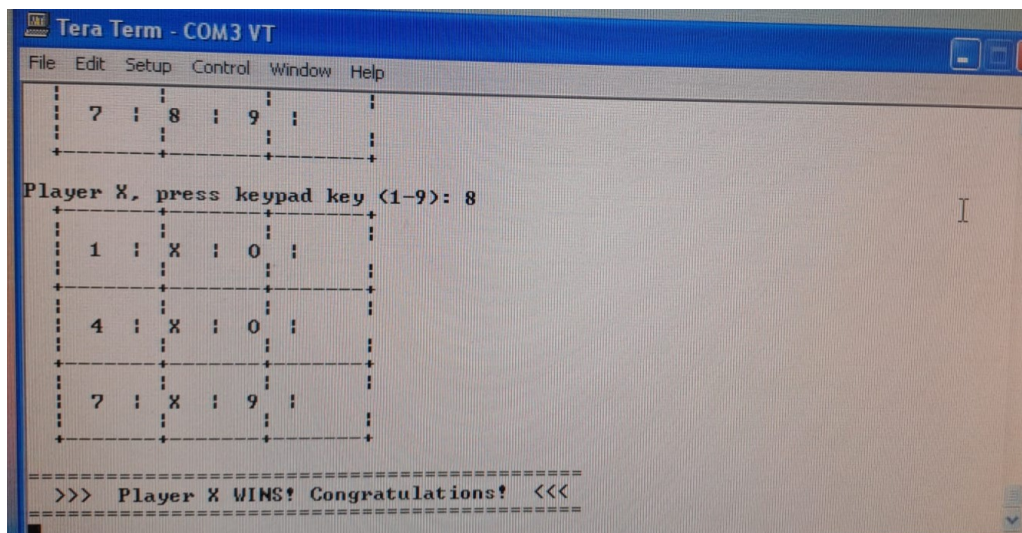


Figure 6.2



Figure 6.3: MTS-86C kit LCD display showing the two-line hardware win message written by `LCD_WIN_MSG`: Line 1 — `PLAYER X WINS!`, Line 2 — `CONGRATULATIONS!`. The player symbol (X or O) is inserted dynamically from the `PLAYER` variable; no hard-coded win string exists for each player separately. The LCD is driven via port `FFC5H` (data) and `FFC1H` (command), with a busy-flag poll on `FFC3H` before every write.

7. Conclusion

The project successfully delivered a fully working Tic-Tac-Toe game in 8086 assembly language, meeting all requirements of the project specification and the bonus hardware integration criteria.

The simulation target (graded out of 5 marks) was achieved through a clean terminal-based user interface over RS-232, correct two-player game logic, complete win and draw detection, and robust input validation. The bonus target (out of 2 marks) was achieved by integrating three physical kit peripherals: the **8279 keypad** for all player input, the **LCD display** for end-game result messages, and the **LED bank** for the win celebration pattern.

The development process reinforced key microprocessors concepts including memory-mapped I/O, PPI configuration, hardware polling, subroutine design, and the flat single-segment programming model used by the MTS-86C trainer kit.

A. I/O Port Reference

The following I/O port map (Figure 1-3 of the MTS-86C manual) lists all relevant ports accessed by the program:

Address	Device	Port	Function
3FD6H	8255-3	CWR	Control Word Register
3FD4H	8255-3	Port C	—
3FD2H	8255-3	Port B	LED output (8-bit)
3FD0H	8255-3	Port A	TACT switch input
FFEAH	8279	Status/Cmd	Keypad status / command
FFE8H	8279	Data	Keypad FIFO data read
FFC5H	LCD	—	Data Write
FFC3H	LCD	—	Instruction (Status) Read
FFC1H	LCD	—	Instruction (Command) Write